

Ask the Tool, Don't Guess: Agent Tool Calls Hold Their Progress, and the Serving System Should Read It

YIPENG LIU, Tsinghua University, China and Alibaba Cloud Computing, China

YINGQIANG ZHANG, Zhejiang University, China and Alibaba Cloud Computing, China

FEIFEI LI, Alibaba Cloud Computing, China

HUANCHEN ZHANG, Tsinghua University, China

An agentic request spends substantial wall-clock time waiting for tools, and its KV cache holds GPU memory the whole time. Serving systems decide whether that cache stays, leaves, or comes back by guessing how long the tool will run, from the tool's name, its history, a duration declared before the call, or the engine's own occupancy. We show that no estimate fixed before a call starts can know its duration, and such estimates may not even rank the calls. Meanwhile, the running tool already holds the answer, but the agent stack together with the tool silences it. We propose that tool calls report their progress explicitly while they run, and we measure what that takes. A census of four public agent corpora finds a readable signal in most tool time once it is revealed, in two strengths: a fraction of the work remaining, or an accurate signal that the end is near. A harness recovers it without changing what the agent sees, at no measurable cost to the agent's benchmark score. At the points where a KV cache decision is made, the reported progress is between several times and an order of magnitude more accurate than the best published predictors, and it stays accurate when the environment changes. Plugged into a production engine through a few small hints, it cuts the p90 time to first token (TTFT) after a tool call by 20.7% (HBM only) and 20.8% (HBM + DRAM) against LRU, close to an oracle. A serving system should not guess what its tools can tell it.

1 Introduction

An agentic request spends substantial wall-clock time waiting for tools [21], and while it waits, its KV cache holds GPU memory that nobody is using. Between two model turns the agent runs a shell command, and the serving system must decide whether the cache stays, leaves to make room, or comes back in time for the next turn. Today it decides by guessing. It reads the tool's name, the durations of past calls, a duration declared before the call, or the occupancy of the engine [1, 6, 7, 14, 25, 34, 44, 50, 56]. None of these looks at the call while it runs. The guess matters most where it is hardest. The waiting is dominated by a few long calls: the installs, builds, test suites, and generated scripts of a coding agent. A few percent of the calls hold two thirds of it. Figure 1 shows one such call from the inside, and what the agent is allowed to see of it.

The guesses fail for a reason that no better model will remove: the information is not available before the call. A long call's duration is set by the load on the machine, a neighbour in the sandbox, or the state of a remote API [5, 9, 13, 16, 43], none of which the command line shows. Predictors built from a call's name, arguments, or history cannot even rank the long calls. History helps only when it was collected in the environment the call now runs in, and a history that is both large enough to trust [25, 50] and collected under one load rarely exists (Section 2.1).

Meanwhile, the running tool already holds the answer, and the stack silences it. The mini-SWE-agent's harness [49] switches progress bars off before the tool starts, and agents pipe five seconds of tool time out of six through tail, quiet flags, and redirects. Even with the silence lifted, the answer is not on the surface: totals are missing, counters stop halfway, and drawing the signal out without changing what the agent sees is most of the work.

Authors' Contact Information: Yipeng Liu, yipeng.liu@tuna.tsinghua.edu.cn, Tsinghua University, China and Alibaba Cloud Computing, China; Yingqiang Zhang, yingqiang.zyq@alibaba-inc.com, Zhejiang University, China and Alibaba Cloud Computing, China; Feifei Li, lifeifei@alibaba-inc.com, Alibaba Cloud Computing, China; Huanchen Zhang, huanchen@tsinghua.edu.cn, Tsinghua University, China.

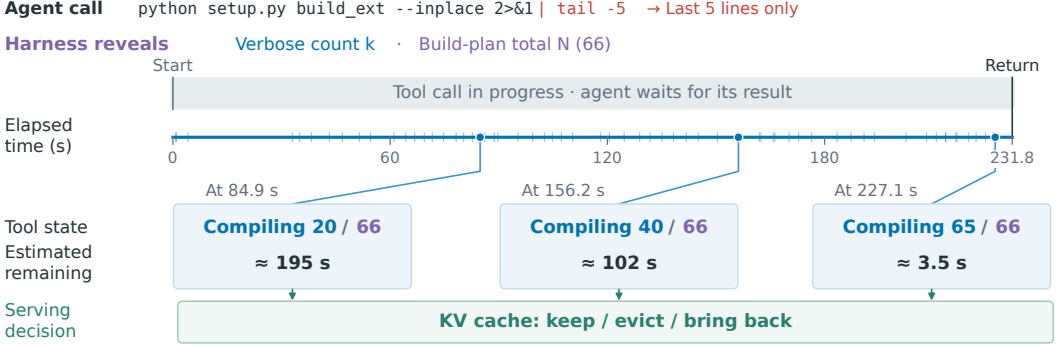


Fig. 1. Progress becomes readable while the tool is still running. A real scikit-learn build (replayed) returns after 232 seconds. The agent’s command pipes the output through `tail` and sees only its last five lines. The harness reads the compiler’s verbose stream, which names each file as it starts, and obtains the total of sixty-six files from a dry run. Three snapshots show the count the tool holds at that moment and the remaining time it implies. These progress reports feed the decision on the KV cache side.

We argue that the serving system should ask the tool instead of guessing. A tool call should report its progress explicitly while it runs, and revise the report as it goes, in place of a duration the system predicts implicitly before the call. What a tool can report comes in two strengths: how much work remains, or only that the end is near. Both are useful to a cache (Section 2.2).

We build this proposal in four steps and measure each one. We take a census of what tool calls hold across four public agent corpora and of what it takes to reveal it (Section 2). We build a harness that recovers the signal without changing what the agent sees (Section 3). We compare the recovered signal with four published predictors at the points where a KV cache decision is made, including under a changing environment (Section 4). We add a few small primitives to a production serving engine and measure what the signal buys on GPUs (Section 5). End-to-end evaluation shows that reported tool progress cuts p90 TTFT after a tool call by 20.7% (HBM) and 20.8% (HBM + DRAM), close to what an oracle achieves.

The paper makes four contributions.

- **The gap, with evidence.** We place today’s timing signals on a ladder, from static caches to declared durations (Table 1), and show why every rung fails: estimates made before the call cannot rank the long calls, and histories are bound to the environment that produced them (Section 2.1, Figures 2 and 3).
- **A census of what tools hold.** We give the first corpus-scale account of progress in agent tool calls: two strengths, five kinds, four sources. We report the share of tool time in each and what it takes to reveal it (Section 2.2, Table 2, Figures 4 and 5).
- **A harness that reads it without changing what the agent sees.** We lift three layers of silence, turn output and environment into one event stream, and measure the result: accurate where the decision is made, free for the agent (Sections 3 and 4, Figures 6 to 8, Tables 3 and 4).
- **What it buys.** We add a few engine primitives, evaluate end to end on GPUs with and without a host memory tier, and bound what a lying session can gain with a session-credit mechanism that treats the tool’s report as untrusted input (Section 5, Figure 9).

In one sentence: agent tool calls already hold their progress, a harness can read it without changing what the agent sees, and a serving system that reads it does better than one that guesses.

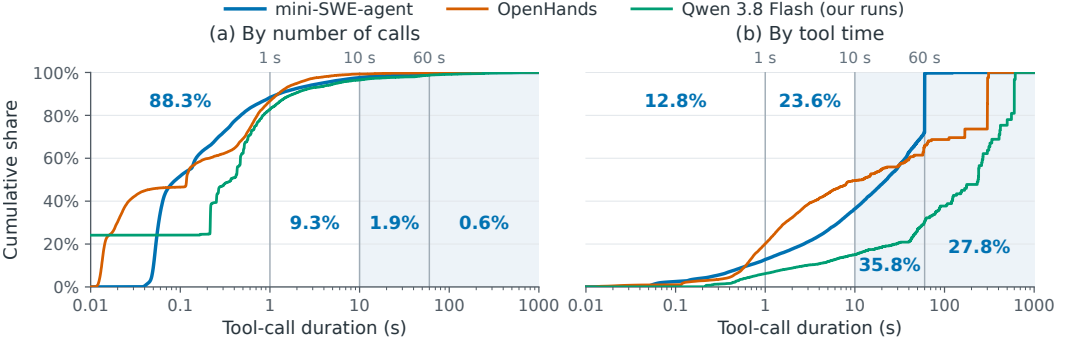


Fig. 2. Long calls dominate tool time. Cumulative share of tool calls (a) by number and (b) by recorded duration, for the leaderboard runs of mini-SWE-agent [37, 49], the OpenHands evaluation runs [53], and our own runs; annotations give the leaderboard’s share in each interval.

2 Duration Is Decided at Run Time, and the Tool Already Holds It

Picture a coding agent that has just typed `python setup.py build_ext --inplace` in a scikit-learn checkout [41] and is now waiting. Somewhere in the serving system a scheduler is asking how long this will take, but all the scheduler has is the command line. The compiler, meanwhile, is on its fortieth of sixty-six files, and it would say so, except that the harness switched the counters off and the agent piped what was left into `tail` (Figure 1).

This section makes two points about that scene. First, no estimate made before the call starts can tell how long it will take. Second, the running tool already holds the answer: sometimes the tool prints the answer and the agent stack hides it, and more often the answer has to be drawn out. Section 2.1 gives the evidence for the first point and places today’s systems on a ladder of timing signals. Section 2.2 is a census of what the tools hold, and of what it took us to reveal it.

2.1 Why guessing fails

The waiting is concentrated in a few long calls. In the SWE-bench [22, 37] leaderboard runs of mini-SWE-agent [49, 58], fewer than three percent of tool calls hold almost two thirds of all tool time (Figure 2): the installs, builds, test suites, and generated scripts a coding agent cannot avoid. Other traces of agent tool calls show the same concentration [56, 62]. For those long calls the retention decision is worth making well; for the sub-second others, almost any policy will do.

The duration of a long call is not a property of the command itself. A call-time estimator can learn the typical duration of a class of calls. However, it cannot learn the duration of this call on this machine. We replayed the commands recorded in leaderboard runs¹ on an idle sandbox. The same command finished an order of magnitude faster (Figure 3a). Restricting the sandbox’s CPU quota changed almost nothing. Adding busy neighbours, however, made most calls several times slower (Figure 3b). The order of the long calls recorded under production load has no correlation with their order on an idle machine, and when the idle machine is oversubscribed, much of its own ordering is lost as well (Figure 3c). Contention is not the only thing outside the command. Package indices, object stores, and model APIs answer at rates that follow the hour and the weekday [5, 16, 20], and their quotas follow the caller’s IP [15]. These patterns of outside providers can hardly be turned into features of a predictor.

¹Throughout we read these leaderboard runs (six submissions, about three thousand trajectories), the OpenHands evaluation runs [53], and our own runs on a hundred random SWE-bench Verified tasks using Qwen 3.8 Flash [42].

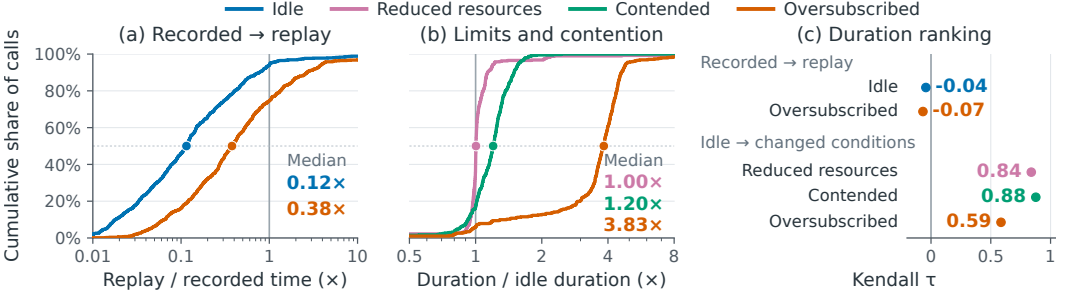


Fig. 3. The same command, different environment. (a) Ratio of replayed to recorded duration for the commands of the leaderboard runs. (b) Ratio of duration under various conditions to the idle replay. Dots mark medians. (c) Rank correlation of the long calls’ durations under various conditions.

What the scheduler reads	Example systems	What it sees
Fixed TTL	CachedAttention [18], SGLang [60], Anthropic [3]	nothing
Elapsed time	InferCept [1], LAMPS [46]	elapsed time
Per-tool statistics	Continuum [25]	name
Learned model	CacheWise [50]	name, arguments
Engine-side occupancy	MORI [56], ConServe [14], ThunderAgent [23], Adaptive KV Retention [12]	engine state, not the call
Declared duration	TokenCake [6], SGLang [44], Mooncake [34], vLLM [52], TensorRT-LLM [36]	fixed estimate
Reported progress	this paper	live progress

Table 1. The ladder of timing signals a serving system consumes during a tool call. No existing rung sees inside the call. This work adds the last rung: a progress report emitted and revised by the running tool.

A history of past calls does help, but only when it was collected in the same environment the call now runs in. A history pooled across environments is an order of magnitude worse near the end of the call than the matching history (Section 4), and a scheduler cannot know which history matches. Moreover, there is rarely enough of it in one place. Continuum, for one, asks for about a hundred past calls per tool before it trusts a distribution [25]. In our corpus only a handful of tool identities reach that count. Serving systems have kept guessing anyway, and their guesses form a ladder (Table 1), ordered by how much of the call each source can see. Static caches keep everything for a fixed time [3, 9, 18, 60]. Elapsed-time estimates keep a call as long again as it has already run [1, 46]. Per-tool statistics turn past calls into a time-to-live [25]. Learned models cluster the arguments [50]. Engine-side observers watch occupancy and idleness [12, 14, 23, 56]. The newest designs let the caller declare a duration once, before the call starts [6, 34, 36, 44, 52].

None of them reads the running call: they all keep predicting its duration implicitly, from what lies around it. Every rung fixes its estimate before the call runs, or never sees inside it. This is an old lesson from another field. Database systems showed long ago that an estimator which never executes the query has no bound on its error [10], and that the only reliable progress indicator is the query’s own count of work done [11, 29]. If the estimate cannot be made before the call, can it be read during it? The same holds here: the missing rung is the one where the running tool reports how far along it is, keeps revising that report while it runs, and guides KV cache decisions on the server side. This is what this work proposes.

Kind and source	One real command	mini-SWE-agent	OpenHands	Ours
Strong signals		38.2	50.6	48.0
A in the output	<code>pytest test_sing... -v OpenHands</code>	13.4	32.9	5.7
B switch or dry run	<code>pytest ... -q tail -12 ours</code>	18.2	11.5	31.8
C a hook	—	0.0	0.0	0.0
D the agent's code	<code>python test_issue.py leaderboard</code>	6.5	6.3	10.4
Weak signals		28.0	8.6	23.8
multi-phase build	<code>make inplace leaderboard</code>	9.6	2.4	0.0
download-install	<code>pip install ... tail -5 leaderboard</code>	10.0	2.9	8.7
count without total	<code>bin/test sympy/vector ... -q ours</code>	8.4	3.2	15.0
other end signal	<code>git stash && pytest ...; stash pop ours</code>	0.0	0.0	0.1
Negligible	<code>sed -i "/codeset/d" trans.py leaderboard</code>	15.5	29.1	6.2
Declared		0.4	0.0	10.1
exact	<code>sleep 420; grep ... test.log ours</code>	0.4	0.0	8.6
bound	<code>timeout 12 django runserver ours</code>	0.0	0.0	1.5
Unparsable	<code>python manage.py dbshell OpenHands</code>	18.0	11.7	11.9

Table 2. What a tool call holds for the serving system. Strong signals report how much work remains. Weak signals report only that the end is near. Shares are percentages of tool time after our instruments are applied.

2.2 What the tool holds, and what it takes to reveal it

The long calls are not silent, they are silenced. The infrastructure removes one layer of signal before the tool starts: every public mini-SWE-agent [49] run sets the environment variables that turn off pip's and tqdm's progress bars. The agent removes a second layer itself. It pipes five out of every six seconds of tool time through `| tail`, `-q`, or a redirect to a file in our own runs. Lifting these two layers is where the work begins. Totals are missing, counters stop halfway through a run, output is buffered until the call returns, and the most informative calls print nothing at all unless asked in the right way. We therefore built the instruments to draw the signal out of each kind of call, and we counted what each instrument yields.

What a tool holds falls into five kinds. Two of them are progress. A strong signal reports how much work remains: a fraction, or a count out of a known total. A weak signal reports only that the end is near: an accurate stop signal, with no fraction worth trusting before it. Two more kinds need no progress at all, because the call is too short to matter or because the command declares its own duration. What is left is unparsable. Table 2 gives a real command for each kind and the share of tool time it holds after our instruments. This section reports the count; Section 3 describes the instruments and shows that the agent never sees them.

Strong signals. They come from four sources, which differ in how far the signal is from the surface. **Source A** is the tool's own output: a test runner prints a percentage column beside its dots, and pip names each package it installs. Reading them needs only the side channel. **Source B** is a switch the tool already has, at its simplest a verbose flag or an environment variable that re-enables a progress bar. It has two harder cases: some switches give a counter but no total. Our scikit-learn build names every file it compiles with `-v` but never says how many there are. Turning the counter into a fraction needs a total obtained without disturbing what the agent reads. In other cases the switch can hardly reach the tool at all. When a package is built in an isolated environment, getting

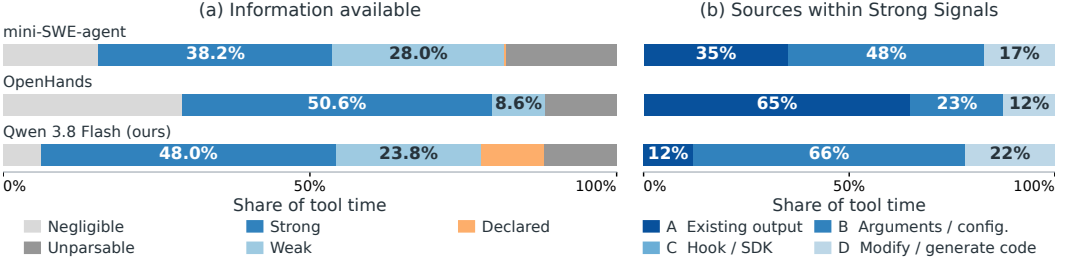


Fig. 4. What tool calls hold after our instruments are applied. As share of tool time, judged after the fact with full knowledge of each call. Table 4 shows what the harness actually delivered.

a verbose flag through to the compiler would take a chain of rewrites in the build scripts. The signal is instead read from the environment, as the object files appear on disk one by one.

Source C is a hook the tool exposes to its host, the way language servers and agent protocols report progress to theirs [31, 32]. It is the cleanest and the rarest source. In all our sampled calls we did not find it used once. Still, it is the form the mechanism should eventually take (Section 3.2).

Source D is code the agent wrote itself: a script generated in the previous turn can be given a progress line as it is generated. The added line prints the script’s progress in the side channel’s standard format. This source calls for generation plus an equivalence check, not for a parser.

Weak signals. Their value is not in the fraction but in the last few seconds. Those seconds are what a KV cache decision needs, and revealing them took a parser of its own. Mainly two families produce this kind of signal. The first includes the multi-phase build, a make-style run that carries out many compilation tasks of very different sizes and then links them. A counter over the tasks is not a clock, and we show that a time extrapolated from it is often unreliable. What is usable is the announcement of the last unit and of the link step after it. We detect both, and they arrive seconds before the call returns, enough to prefetch a context back into GPU memory or to recompute it. The second family is the download, for example `pip install`. Most installs are sub-second, but the ones that stall on the network are the ones a scheduler cares about. Of the installs that stall, those that print at all announce their final phase about a second before they return. A weak signal cannot say how much is left, but in most cases its announcement of the final phase comes early enough to start a reload or a recompute in time.

The remaining kinds. They do not need progress, and we classify them before the call runs. Calls too short to matter can be recognised from the command text alone with high precision. The few that turn out long do so because of the environment, not the command, and a short timer catches them. Calls that declare their own duration give the scheduler a deadline rather than an estimate: a sleep is exact, and so is a timeout wrapping a command that would never return on its own; around any other command the timeout is an upper bound, which is what a prefetch needs. Calls that cannot be parsed are the boundary of the approach, for example a long remote API call, or a command that waits for human feedback.

The revealed signals have the shapes a scheduler needs (Figure 5). A test suite read from its own output is linear after a short collection phase, with the unevenness of individual tests as noise. A build read under its verbose flag, with the total from the dry run, is linear across almost the whole run. Isolated builds, read from the object files on disk, have the verbose build’s shape. If read from its default output, the same build only shows the Cython phase, which does not represent the whole progress. An instrumented script is linear to the last event.

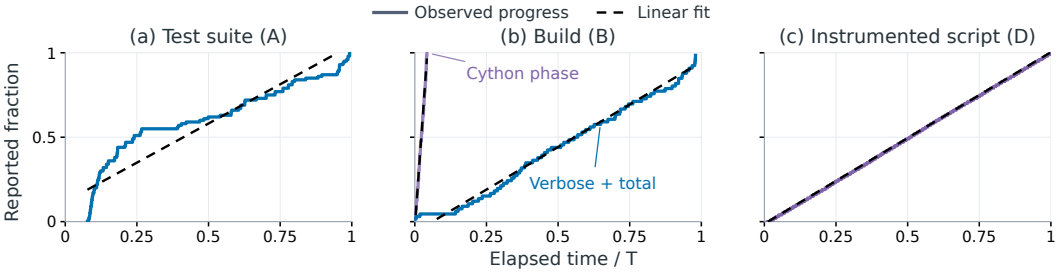


Fig. 5. What the revealed signals look like. Reported fraction versus elapsed time for one long call of each kind. Dashed lines are linear fits over the observed progress stage.

How much the agent hides depends on the model. Under the same harness and the same tasks, a different model may suppress more of its tool output and leave less of the long calls readable, but the tools hold the same things for every model. What varies is how much the agent throws away, not whether the signal exists. The tool holds it. With every instrument applied, a strong signal covers 38% of the leaderboard's tool time, 51% of the OpenHands runs', and 48% of our own. A weak signal raises that to 66%, 59%, and 72% (Figure 4a). The following section shows how the harness draws it out, and how an extendable execution wrapper reads it.

3 Reading It Without Changing What the Agent Sees

Back to the scikit-learn build. The compiler is on its fortieth file, and our harness has one job: get that sentence to the scheduler while the agent keeps reading exactly the bytes it would have read anyway. This section describes how. First, the rule that governs the design and the path the signal takes from the tool to the scheduler. Second, the three layers of silence, and what lifts each. Third, the wrapper that turns tool output and the tool's footprint on disk into one stream of progress events. Last, how the same design extends to tools the base model has no knowledge of.

3.1 Three layers of silence, and what lifts each

One rule governs everything in this section: the agent's observation should not change. The result path to the agent is therefore untouched, and progress leaves the execution environment by a second, separate path (i.e., a side channel). Figure 6 shows the two paths of one execution: the unchanged result path to the agent, and the state path to the serving system. Before a tool call starts, the harness registers it under a call id with the serving side and prepares the command where needed: a switch, a dry run for the total, or an instrumented script. It then executes the tool. While the tool runs, an executor-side tap reads two things: the stream itself, or a side copy of it, and the files the tool leaves on disk. It reports timestamped counts, totals, and phase markers under the call id, and the serving side decides from the remaining-time estimate what to do with the request's KV cache. When the tool returns, its output and exit status reach the agent in their original formatting. Every piece lives in the harness with minimal modification to the serving engine.

The first layer of silence comes from the infrastructure and from the agent's own habits. Our harness restores the progress bars that the environment variables had turned off, and strips them again from the stream the agent reads, so the bars exist only on the side channel. A `-q` flag is reversed in the same way. Where the agent cut the output with `tail`, or sent it into a file, the harness duplicates the full stream into a side file inside the execution environment. `tail` still reads the same last lines, the file is unchanged, and so is the exit code. Section 4 also measures the variant

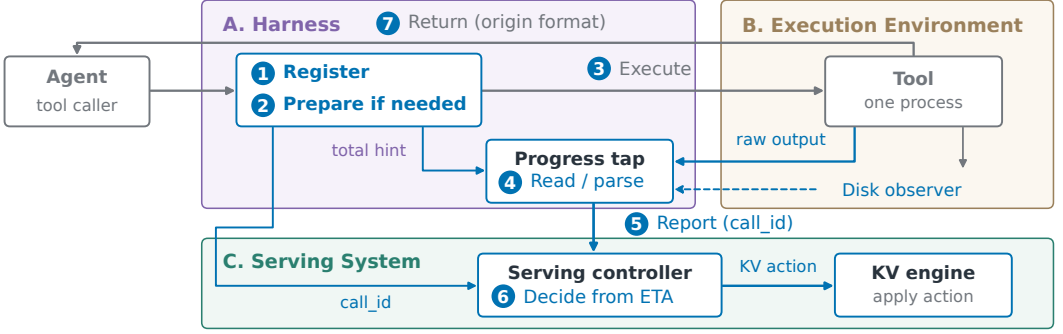


Fig. 6. One execution, two consumers. The harness registers the call, prepares the command when needed, and executes the tool. While the tool runs, a tap reads its output and the files it leaves on disk, and reports its progress under the call id. Grey is the unchanged result path; blue is the state path this paper adds.

in which the agent simply sees the full output. Together these rewrites put about two thirds of the suppressed tool time back on the side channel.

The second layer of silence is that many tools speak only when asked in the right way. Lifting it means asking on the agent’s behalf and keeping the answer on the side channel. A verbose flag makes the scikit-learn build name every file it compiles, but not how many files there are. The harness first runs the build’s own dry run to obtain the total, then adds the flag. The verbose lines go to the side channel and are removed from the stream the agent reads, so the agent’s stderr is byte for byte what it was. When the flag cannot reach the compiler (e.g., in an isolated build), the wrapper watches the build directory instead. It counts the object files as they appear, with the total taken from the package’s extension list. Test runners need no flag at all. They already print one dot per test as it finishes, without a newline. The wrapper parses the partial line as it grows, so the fraction arrives test by test instead of at the end.

The third layer is the agent’s own code, and lifting it is a generation task rather than a parsing task. A script the agent writes usually has no progress output. The harness has the model add it at generation time: a progress line inside the loop, emitted in the side channel’s format, with the requirement that the script’s behaviour is otherwise unchanged. We check that requirement in two ways. We run the original and the instrumented script and compare their output and exit status, and we compare the agent’s benchmark scores with and without instrumentation (Section 4). The progress lines themselves never enter the agent’s observation. Table 5 in Appendix B gives detailed examples of the instruments by layer of silence.

3.2 One event stream from the wrapper, and how it extends

The wrapper sits between the executor and the tool process. It passes the tool’s bytes through unchanged, and on the side it produces events. It has two layers. One reads what the tool prints; the other reads what the tool leaves behind in the execution environment. Both produce the same kind of event, and both send it over the same side channel keyed by the call id.

The first layer turns a byte stream into events. Every event has the same shape: a call id, the work done, the total if known, the phase, and a timestamp. A registry of parsers recognises the counters and percentages of the common tools, a generic parser catches the rest, and a shim makes progress-bar libraries emit structured lines even when their display is disabled. A separate parser recognises the end markers of a tool’s last phase. The second layer extends the byte stream. It reads the environment rather than the output. The object files of a build appear on disk one by

Corpus	Precision (%)	Recall (%)	Sent short (% of calls)	Wrongly sent, ≥ 10 s (% of time)
mini-SWE-agent	98	80	72.5	2
OpenHands	97	83	74.1	15
Ours	96	58	47.9	2

Table 3. Sorting calls before they run. Precision and recall of the sorting by the harness, the share of tool calls it sent as negligible, and the share of tool time in the calls it wrongly sent.

one. The size of a download is recorded in the package index before the download starts. A test runner’s collection phase leaves a count. Whatever trace a tool leaves of its work in the execution environment traces its progress, and the wrapper can read it without the tool saying a word. This is how isolated builds become as readable as verbose ones. Together, the two layers add less than one percent to the wall clock of a realistic call.

Two things make the stream trustworthy rather than merely present. When a counter passes the total it was given, the wrapper retracts the report and falls back to a count without a total, rather than clamp at one hundred percent and send false alarms. And a call with several phases is not left as several disconnected bars. Where earlier runs of the same command show how much of the call each phase takes, the wrapper folds the bars into one fraction with those shares as weights, and the call becomes a strong signal. Where the shares are unknown or unstable, the wrapper keeps only the last bar and reports its end as a stop signal, and the call stays a weak one.

What the base model can describe from pre-training already covers most of the tool calls we observed. Three paths extend the coverage to tools it has no knowledge of. The first is to teach the agent. For a given command, our harness can tell the model which switch turns progress on and supply the matching parser, and the agent then asks for progress itself. The second path belongs to tool owners. A tool can expose a hook to its host and report its own progress through it [31]. The Model Context Protocol defines a progress notification for exactly this purpose [32, 33]. What has been missing is a consumer on the serving side. The third path is automatic. A harness that records which tool calls hold the most time can scan them in the background, try their switches, and add the ones that work to its rules.

4 How Good Is It, and What Does It Cost?

Now the progress signals are there. The question a production system asks next is how good they are, and what they cost the agent. A KV cache decision has two ends. When memory runs short in the middle of a tool call, the system must decide whether this context is worth evicting. Near the end of the call it must decide when to start bringing the context back, and whether it is already too late. This section measures the signal at both ends: how much tool time can be sorted into its class before a call starts, and how the progress stream compares with four published predictors at those critical points. We also measure which degrades when the environment changes, what the agent paid, and how the picture changes across base models.

4.1 Before the run

Sorting calls before they run. Most tool time can be sent down the right path before the call starts. A lightweight classifier decides whether a call is too short to matter, declares its own duration, needs progress, or cannot be parsed. Its precision on the tool calls is high, and the calls it misses are long because of the environment rather than the command. A short timer that re-registers any call that has not returned catches them. Table 3 reports the precision and recall of that sorting per corpus, and the share of tool time in the calls it wrongly sends down the short path.

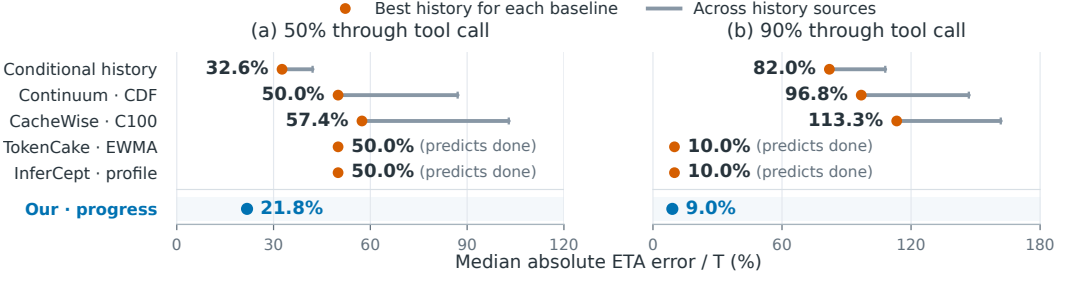


Fig. 7. Remaining-time error of each estimator. The median absolute error relative to the call’s duration. For each published predictor the dot is its best across history sources and the bar spans all (see Section 4.3). InferCept’s elapsed rule is ignored by construction; the two static predictors read “done” at both points.

The competitors. We compare against four published predictors, each reimplemented as its paper defines it: **Continuum**’s [25] per-tool time-to-live from the empirical distribution of past durations, **CacheWise**’s [50] clusters of tool names and arguments, **TokenCake**’s [6] running average of recent calls, and **InferCept**’s [1, 46] two rules, a profiled duration per tool and the elapsed rule that a call has as long left to run as it has already run. Progress signals from our instruments are converted linearly into a remaining-time estimate. We also add our own in-flight history estimator in the manner of 3Sigma’s conditioning on elapsed time [39].

4.2 Accuracy at critical KV decision points

Halfway through the call (whom to evict). Memory runs short for reasons outside any one call. From the call’s point of view the moment is random: uniform over the call, and on average halfway through. We judge the eviction decision at the midpoint. Among the idle contexts resident in GPU memory, the serving system evicts the one whose call has the longest to run, and must not evict one whose call is about to return, because the reload would arrive too late. Both are questions about the remaining time. As a courtesy, we fix the sandbox configuration for the predictors, where their history matches the current machine. The changing-environment case comes below. Across all the methods at 50% of each call, the progress stream’s median error is about a fifth of the call’s duration, against a third for the best history predictor (Figure 7a).

At the end of the call (when to bring it back). Near the end, the progress stream is far more accurate than any predictor, because the history predictors are reading the far end of a distribution that no longer describes this call. A call still running at that point has outlasted most of its class, and the history has little left to say about it. The progress stream’s median error at 90% of each call is under 10%, against 80% for the best history predictor (Figure 7b). The two static predictors that always answer “almost done” look as good as the stream at 90% by construction, and they raise a false alarm at nearly every earlier checkpoint. We further quantify that error is not the whole story. What a scheduler needs is a trigger at the right moment, and that is a stricter test. The recovery budget is defined as the time it takes to get a context back, and an estimator triggers when it first reports a return within that budget. The trigger is timely if it lands just enough budget before the return. Later causes the next turn to stall, while too early puts the context in GPU memory for nothing. Figure 8 uses two realistic budget settings. A late trigger helps nothing, so we count the runs whose first trigger is timely as a function of the extra lead time allowed. The progress stream is timely in about a quarter of the runs with a two-second budget and a sixth with a half-second budget, far better than the baselines. The appendix breaks these down by family, where uniform units do markedly better than uneven tests across tool calls.

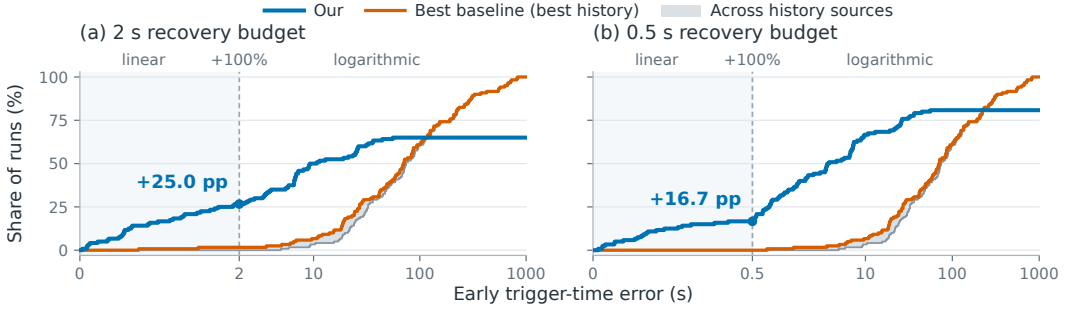


Fig. 8. Timely first triggers. A run counts as timely when the estimator first reports a return no later than the recovery budget before the return, and no more than an extra budget early. The curves give the share of runs as a function of the extra lead allowed. Blue is the progress stream. Orange is the best predictor with its best history, and the band spans all history sources (Section 4.3).

4.3 Robustness across settings

Across environments. Guesses degrade with the environment; the progress stream does not. We replayed the same calls on the same machine under four conditions: idle; contended throughout; contended from a third of the way in; and contended until the halfway point. When we feed a predictor with matching history it may be usable, while a pool of mixed histories makes it an order of magnitude worse near the end. In contrast, the progress stream is equally accurate across the environments, because it does not depend on history. It reads the call’s own rate, so a change of machine costs it only the few seconds until the new rate is observed. The matrices and the trajectories after a load change are in Appendix C.

Across harnesses. Besides the production harness, we also measure the side channel alone to see what each addition brings. On our sample of SWE-bench Verified tasks, neither changes the agent’s score significantly, and the side channel goes further: steps, tokens, tool time, and wall time are all unchanged as well (Table 4). Furthermore, equivalence rests on the observations being byte-identical. A stock harness already exposes some signal. On top of that, the side channel delivers a strong signal on a fifth of the tool time, and the prompt raises that to a third, at the price of output tokens. We also tried a stronger, more intrusive variant that removes the suppression from what the agent sees. De-suppression changes the exit codes of many calls, posing a risk to the agent’s behaviour. However, it does not raise the signal share at all. The longest test runs sit behind a grep or a sort in the middle of a pipeline, which no rewrite of the command’s tail can undo.

Across base models. The harness is model-agnostic: the same parsers, switches, and dry runs serve every model with no per-model tuning, so the signal a tool holds is the same whichever agent runs it. What changes with the model is how much of that signal survives the agent’s habits. We ran the side-channel configuration on the same tasks with two open-weights models [42] and two closed frontier models [2, 38], each at its highest reasoning budget. They differ in how they work far more than in what they solve: one frontier model solves as many tasks as the open ones in half the steps, verifying its work throughout; the other submits after a handful of steps and rarely verifies. They differ in what they hide, too. The share of long-call time on which the harness recovers a usable stream ranges from about a third to about half across models, and follows how much output the agent suppresses, not how strong the model is.

	stock	side channel	+ prompt	de-suppressed
Strong signal (% of tool time)	–	+20.0	+33.3	+15.3
B, switch or dry run	–	+19.9	+17.5	+15.2
D, the agent’s code	–	+0.1	+15.8	+0.1
Weak signal (% of tool time)	–	+10.2	+7.6	+18.8
	mean	paired difference vs. stock		
Solved (of 100 tasks)	100	–1	–2	–1
Steps	35.9	+0.40	+2.11	–0.37
Completion tokens	14,764	–458	+1,575	+70
Tool time (s)	205.1	–15.2	–47.2	–90.3
Wall time (s)	495.8	–34.9	–24.0	–82.7
Non-zero exit codes	298	–34	+22	+455

Table 4. Behaviours and costs in each harness configuration. Upper rows: the share of tool time on which a strong or a weak signal reached the serving side under each configuration; the stock configuration has no tap. Lower rows: the agent’s outcome and cost under stock, and how each configuration changed them, bold where the 95 % bootstrap interval excludes zero.

5 What the Signal Buys the KV Cache

The signal is accurate where it matters and free where it should be. We now examine its contribution to KV cache decisions. With GPU memory alone, the only decision is which idle context stays when memory runs short. With a host tier below the GPU, two more decisions appear: when to let an idle context go to the host, and when to bring it back. Both take the same signal, and the engine needs only a few hints to use it, which we added to the vLLM engine [24] in a few hundred lines (Appendix D). Section 5.1 sets up the replay and the two memory configurations, Section 5.2 reports what the signal buys end to end, and Section 5.3 asks what happens when a reporter lies.

5.1 Setup and baselines

We replay the agent sessions of Section 4 across all methods on a 4×H100 SXM instance in two configurations: GPU memory only, and GPU memory with DRAM as an offload tier. We include two more baselines: the engine’s own LRU, and an oracle that knows every exact return time. The two configurations pose different decisions. With GPU memory alone, a wrongly kept context costs someone else a recompute and a wrongly evicted one costs this call a recompute, so the decision is a pure ordering: who stays. With a host tier, an evicted context is copied to DRAM rather than dropped, so a wrong let-go costs only a reload, and the decision becomes when to let go and when to fetch back, paid for mainly in latency and DRAM traffic. Our method based on progress keeps a context while the reported remaining time is under a threshold. With a host tier it adds hysteresis, letting go only once the estimate has moved well past the threshold, and refreshes the context one lead time before the predicted return.

5.2 End to end

Figure 9 shows the change in post-tool TTFT p90 and in throughput against LRU. With GPU memory alone, ordering by reported progress cuts the p90 TTFT after a tool call by a fifth and raises throughput slightly; the oracle cuts it by a little over a quarter. With a host tier, progress cuts the p90 by 21% and the oracle by 23%, and both reload a third of what LRU does. The four predictors collapse into two behaviours, and neither reads the call. Continuum’s per-tool time-to-live does not expire for many calls, where it issues no hint and falls back to LRU. CacheWise, TokenCake and InferCept estimate each call from the short-dominated history of its kind, so their estimates

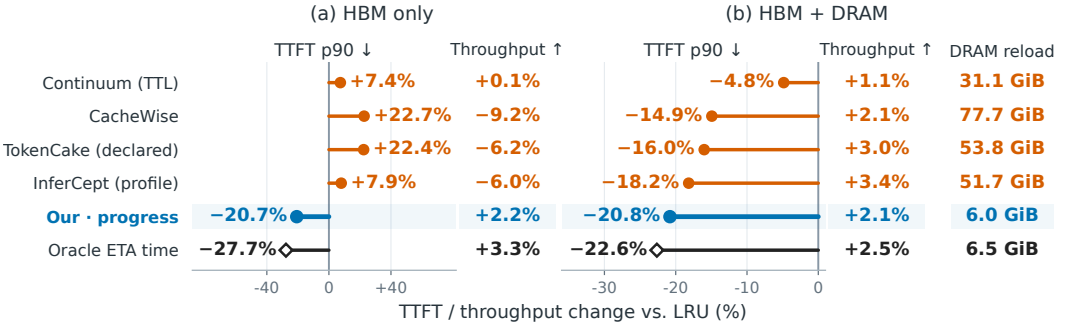


Fig. 9. Post-tool TTFT and throughput in two memory configurations. (a) GPU memory only; (b) GPU memory with a host DRAM tier, with the volume each run reloads from DRAM. Blue is our method.

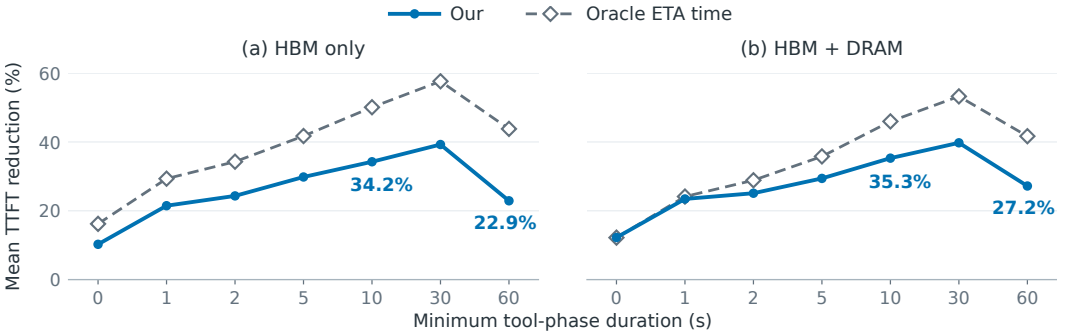


Fig. 10. Mean post-tool TTFT reduction against LRU for tool calls of at least the given length, in both memory configurations. Short calls gain little; the gain grows with the length of the call, reaching 34.2% and 35.3% at ten seconds or more, and the oracle pulls ahead as the calls get longer.

fall below the keep threshold for most calls and they keep nearly everything. Keeping everything holds the contexts of the long calls, exactly the ones that will not return soon, in the memory that live turns need. With GPU memory alone that costs a p90 up to a quarter worse than LRU and a lower throughput. With a host tier it cuts the p90 by 15 to 18%, but the estimates run out early, so the controller fetches contexts back from DRAM before they are needed, at eight to thirteen times our method's reload volume.

Figure 10 breaks the gain down by the length of the tool call that preceded it, as the mean post-tool TTFT reduction on the calls of at least a given length. On the whole cohort the mean gain is about a tenth, because most calls are short and nothing is evicted within a second. The gain grows with the call: about a quarter from two seconds, a third from ten, and 40% from thirty, in both configurations. The oracle pulls ahead as the calls get longer, reaching half at ten seconds with GPU memory alone; with the host tier the two stay closer. On the few calls of a minute or more, ours falls back to a quarter while the oracle keeps most of its gain. A per-phase replay of the decisions splits the remaining distance to the oracle. Giving our method the truth only on the phases it never hears from recovers 63% of the oracle's prize; giving it the truth only where it already has events recovers 17%. The uncovered channel is worth about four times the covered one, so the next lever is coverage rather than precision: every step of coverage moves our method towards the oracle while the wasted refreshes fall.

5.3 When the reporter lies

A progress report is produced by content the tenant controls, so the serving system must assume that a session can lie: a script can do its heavy work last and report ninety percent done [10, 61], and nothing in the transcript reveals it before the call returns. We therefore give no report authority on its own. Each session earns credit from reports that turned out to be right, spends it whenever the engine acts on a report, and is charged when the tool returns and the report is found wrong [19, 51]. Two ledgers exist, one for memory kept on a report's word and one for latency risked on it, so a lie can burn only the ledger it draws on. A session with no credit left is served the way the engine serves everyone without a signal [4, 55], so the worst a liar can do is lose the benefit, and honest sessions are unaffected. In a preliminary simulation, lying raised the memory the liars held by 6%, and the two ledgers took 5% of it back. In conclusion, the tool holds the signal, the harness draws it out without changing what the agent sees, an engine needs only a few hints to use it, and liars do not affect the system much.

6 Related Work

Table 1 already places the systems on one ladder by their timing signal. This section covers the three neighbours a reader will check first, then what is already known and what we add.

Declared durations. TokenCake [6] and the recent engine proposals that carry a field such as an expected tool duration [34, 36, 44, 52] already let the caller state how long a tool will run, and time offload and upload against it. Every such value is fixed before the tool starts and can be corrected only when it returns. We keep that machinery and replace the constant with a stream the tool emits while it runs. The difference is measured rather than argued.

Predicted durations. Continuum [25], CacheWise [50], and InferCept [1] predict a tool's duration from its type, its arguments, or its history. CacheWise and InferCept measured how far such prediction sits from an oracle, on sandboxes of one fixed size. They also need a history that is large and well matched. Continuum asks for about a hundred past calls per tool before it trusts its distribution, and a history collected under one load is of little use under another. Our claim is about the decisions that need a remaining time, and about the calls whose duration is set by the environment, which no call-time feature can see.

Engine-side observation. MORI [56] and ConServe [14] act on what the engine can see, occupancy and idleness, and ThunderAgent on phase markers the harness declares [23]. None of them takes anything from the tool itself. Occupancy cannot tell a call that has just started from one that is almost finished. Our signal is a count of work already done, read from the tool itself.

What is already known.

- Tools can express information to the serving system [40, 57].
- A channel for explicit tool progress exists in the agent protocol [32, 35].
- KV caches accept hints [36, 45, 52].
- Tool durations are heavy-tailed [26, 62], and an estimator has no error bound [10].
- Per-client accounting of KV cache exists [8, 47, 54, 59].

What we add.

- The idea of reporting progress explicitly instead of predicting it implicitly, carried through to the first serving-side consumer of an in-flight, revisable progress report.
- The first census of how much progress agent tool calls hold and what it takes to reveal it.
- The treatment of a tool-originated timing signal as untrusted input.

7 Conclusion

The calls that make a serving system wait are the ones whose duration cannot be known before they start, and they are also the ones that know it best. This paper measured both halves of that sentence. Call-time estimates lose the order of the long calls, and histories are bound to the environment that produced them. Meanwhile, the tools already print or can be made to reveal how far along they are, in two strengths, and a harness can read them without changing what the agent sees. Read at the moments when a KV cache decision is made, the signal is more accurate than any published predictor, and it stays accurate when the machine gets busy. A production engine then turns it into latency and throughput. What limits the gain today is coverage, and we call on tools to expose their own state for the cache to read. A serving system should not guess what its tools can tell it.

Limitations

Three boundaries of this work point in the same direction.

Some tool calls hold no information. A remote API that answers only at the end, a command that waits for a person, and a download stalled before its first byte hold nothing a harness can read, and a harness made of such calls is the boundary of our census. What would move it is on the tool side: streaming and chunked responses from the services, and a progress notification from the tools that already know their own state [27, 28, 32].

Some information cannot be parsed. Our wrapper reads what the common tools print and what they leave behind in the execution environment, but a tool with an output format we have not met, a build system that hides its units, or a script that reports in its own words is opaque to it until a parser exists. The three extension paths of Section 3 reduce this set; they do not close it.

Drawing the signal out may change the tool. A switch, a dry run, or an instrumented script is a change to how the tool runs, even when the agent's observation is unchanged. We measured the cases we used and found them within noise, but every new instrument has to be measured again, and a tool that behaves differently under its verbose flag would break that invariant.

Each boundary is a place where the tool could help. A tool that exposed its progress, in a form it chose, would need no switch, no dry run, and no parser; the harness would only relay it. That is the direction we see: tools that expose their own state, and agent systems built to be friendly to the caches that wait for them.

Ethics Statement

Three consequences of this design should be stated. A tool's progress report is produced by content the tenant controls, so a session can lie; the credit mechanism makes lying expensive rather than impossible. A stream of progress reports reveals the structure of a tenant's workload to the serving system [48, 54], and a deployment should treat that side channel as sensitive as the tool output itself. And when the serving system misjudges a call, the cost falls on the session that waits or on the sessions evicted to make room; the credit ledgers decide who pays, and that allocation is a policy choice, not a technical necessity. Our traces come from public benchmarks and from runs of open-weights and commercial models on them; no user data is involved.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. <https://proceedings.mlr.press/v235/abhyankar24a.html>
- [2] Anthropic. 2026. Claude Fable 5.1 and Claude Mythos 5.1 System Card. <https://www.anthropic.com/claude-fable-and-mythos-5-1>.
- [3] Anthropic. 2026. Prompt caching (cache_control ttl 5m / 1h). <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>.
- [4] Ziyad Benomar, Romain Cosson, Alexander Lindermayr, and Jens Schlöter. 2025. Non-Clairvoyant Scheduling with Progress Bars. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 93520–93561. arXiv:2509.19662 [cs.DS] doi:10.52202/085713-3127
- [5] Song Bian, Minghao Yan, Anand Jayarajan, Gennady Pekhimenko, and Shivaram Venkataraman. 2025. What Limits Agentic Systems Efficiency? arXiv:2510.16276 [cs.AI] <https://arxiv.org/abs/2510.16276>
- [6] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. 2027. TokenCake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. In *Proceedings of the European Conference on Computer Systems (EuroSys '27)*. arXiv:2510.18586 [cs.DC] doi:10.48550/arXiv.2510.18586
- [7] Anish Biswas, Kanishk Goel, Srivashine S, Jayashree Mohan, Alind Khare, Anjaly Parayil, Ramachandran Ramjee, and Chetan Bansal. 2026. Sutradhara: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference. arXiv:2601.12967 [cs.DC] doi:10.48550/arXiv.2601.12967
- [8] Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. 2025. Locality-aware Fair Scheduling in LLM Serving. arXiv:2501.14312 [cs.DC] doi:10.48550/arXiv.2501.14312
- [9] Chaokun Chang, Yukun Zhou, Kaihua Fu, Dakai An, Tianyu Feng, Hanfeng Lu, Sheng Yao, Pu Guo, Yinghao Yu, Yizhou Shan, Bo Li, Binhang Yuan, and Wei Wang. 2026. From LLM Inference to Agentic Workloads: Characterization and Implications for Serving Systems. arXiv:2608.15127 [cs.OS] <https://arxiv.org/abs/2608.15127>
- [10] Surajit Chaudhuri, Raghav Kaushik, and Ravishankar Ramamurthy. 2005. When Can We Trust Progress Estimators for SQL Queries?. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*. ACM, 575–586. doi:10.1145/1066157.1066223
- [11] Surajit Chaudhuri, Vivek Narasayya, and Ravishankar Ramamurthy. 2004. Estimating Progress of Execution for SQL Queries. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. ACM, 803–814. doi:10.1145/1007568.1007659
- [12] Minseo Choi and Ananya Joshi. 2026. Adaptive KV Retention for LLM Agents at Human-Approval Timescales. arXiv:2608.30830 [cs.OS] <https://arxiv.org/abs/2608.30830>
- [13] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80. doi:10.1145/2408776.2408794
- [14] Jianru Ding, Ryien Hosseini, Pouya Mahdi Gholami, Mingyuan Xiang, and Henry Hoffmann. 2026. Observation, Not Prediction: Conversation-Level Disaggregated Scheduling for Agentic Serving. arXiv:2606.01839 [cs.DC] doi:10.48550/arXiv.2606.01839
- [15] Docker, Inc. 2026. Docker Hub Pull Usage and Limits. Docker Docs. <https://docs.docker.com/docker-hub/usage/pulls/> Accessed 14 September 2026.
- [16] Dominik Durner, Viktor Leis, and Thomas Neumann. 2023. Exploiting Cloud Object Storage for High-Performance Analytics. *Proceedings of the VLDB Endowment* 16, 11 (2023), 2769–2782. doi:10.14778/3611479.3611486
- [17] Exgentic. 2026. Exgentic/agent-llm-traces: OpenTelemetry traces of LLM agent runs across benchmarks. <https://huggingface.co/datasets/Exgentic/agent-llm-traces>. Accessed 14 September 2026.
- [18] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [19] Isaac Grosf and Michael Mitzenmacher. 2022. Incentive Compatible Queues Without Money. arXiv:2202.05747 [cs.GT] doi:10.48550/arXiv.2202.05747
- [20] Alexandru Iosup, Nezih Yigitbasi, and Dick H. J. Epema. 2011. On the Performance Variability of Production Cloud Services. In *11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid 2011)*. IEEE Computer Society, 104–113. doi:10.1109/CCGrid.2011.22
- [21] Jiabao Ji, Yujian Liu, Li An, Rohit Jain, Gungor Polatkan, Siyu Zhu, and Shiyu Chang. 2026. Speculate While You Reason: Teaching Agents to Predict Their Next Tool Call via Joint Agent-Speculator RL. arXiv:2607.25816

- [22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR 2024)*. 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- [23] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. 2026. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. arXiv:2602.13692 [cs.OS] <https://arxiv.org/abs/2602.13692>
- [24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*. ACM, 611–626. doi:10.1145/3600006.3613165
- [25] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS] doi:10.48550/arXiv.2511.02230
- [26] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. 2026. Agentic Coding in the Wild: Characterizing GitHub Copilot Traces at Production Scale. arXiv:2608.00101 [cs.AI] <https://arxiv.org/abs/2608.00101>
- [27] Gang Luo. 2017. Toward a Progress Indicator for Machine Learning Model Building and Data Mining Algorithm Execution: A Position Paper. *ACM SIGKDD Explorations Newsletter* 19, 2 (2017), 13–24. doi:10.1145/3166054.3166057
- [28] Gang Luo, Tong Chen, and Hao Yu. 2007. Toward a progress indicator for program compilation. *Software: Practice and Experience* 37, 9 (2007), 909–933. doi:10.1002/spe.792
- [29] Gang Luo, Jeffrey F. Naughton, Curt J. Ellmann, and Michael W. Watzke. 2004. Toward a Progress Indicator for Database Queries. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. ACM, 791–802. doi:10.1145/1007568.1007658
- [30] Mike Merrill, Alexander Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong (Ryan) Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Rytting, Ryan Marten, Yixin Wang, Jenia Jitsev, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (Eds.). 40903–40986. https://proceedings.iclr.cc/paper_files/paper/2026/file/444a3737adaee10d86ad2ef5f74468e6-Paper-Conference.pdf
- [31] Microsoft. 2022. Language Server Protocol Specification, Version 3.17. Specification, version 3.17.0 (10 May 2022). <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/> Accessed 14 September 2026.
- [32] Model Context Protocol contributors. 2025. Model Context Protocol Specification (revision 2025-06-18): Progress. MCP specification, notifications/progress. <https://modelcontextprotocol.io/specification/2025-06-18/basic/utilities/progress> Accessed 14 September 2026.
- [33] Model Context Protocol contributors. 2025. Tasks (SEP-1686), Model Context Protocol specification revision 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25/basic/utilities/tasks>. Accessed 14 September 2026.
- [34] Mooncake contributors. 2026. [RFC]: Agent-Aware KV Cache Support in Mooncake (Phase 1) (Mooncake issue #2098). <https://github.com/kvcache-ai/Mooncake/issues/2098>. Accessed 14 September 2026.
- [35] NVIDIA. 2026. Agent Tracing: Export Dynamo request traces, tool-call metadata, and Perfetto timelines. <https://github.com/ai-dynamo/dynamo/blob/main/docs/fern/pages/use-cases/agents/agent-tracing.md>. Accessed 14 September 2026.
- [36] NVIDIA. 2026. TensorRT-LLM KV Cache System: KvCacheRetentionConfig and TokenRangeRetentionConfig. NVIDIA TensorRT-LLM documentation. <https://github.com/NVIDIA/TensorRT-LLM/blob/main/docs/source/features/kvcache.md> Accessed 14 September 2026.
- [37] OpenAI. 2024. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>. Accessed 14 September 2026.

- [38] OpenAI. 2026. GPT-6 Astra System Card. <https://deploymentsafety.openai.com/gpt-6-astra>. Accessed 14 September 2026.
- [39] Jun Woo Park, Alexey Tumanov, Angela Jiang, Michael A. Kozuch, and Gregory R. Ganger. 2018. 3Sigma: Distribution-based Cluster Scheduling for Runtime Uncertainty. In *Proceedings of the Thirteenth EuroSys Conference (EuroSys 2018)*. ACM, 1–17. doi:10.1145/3190508.3190515
- [40] R. Hugo Patterson, Garth A. Gibson, Eka Ginting, Daniel Stodolsky, and Jim Zelenka. 1995. Informed Prefetching and Caching. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP 1995)*. ACM, 79–95. doi:10.1145/224056.224064
- [41] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [42] Zihan Qiu, Zekun Wang, Xiao Li, Yanpeng Li, Yang Xu, Yixuan Wang, Huaqing Zhang, Rui Men, Bochao Mao, Chengruidong Zhang, Fan Zhou, Hao Luo, Haofeng Huang, Haoran Lian, Haoyan Huang, Hongqing Chen, Jianwei Zhang, Jing Xu, Junjie Wang, Langshi Chen, Liangyu Wang, Linlang Jiang, Man Yuan, Minmin Sun, Peng Jin, Siqi Zhang, Siyu Wang, Xingzhang Ren, Yakai Wang, Yi Zhang, Yiming Dong, Yizhong Cao, Yubo Ma, Yunfei Mao, Bo Zheng, and Dayiheng Liu. 2026. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability. arXiv:2608.30320 [cs.CL] <https://arxiv.org/abs/2608.30320>
- [43] Gian Segato. 2026. Quantifying Infrastructure Noise in Agentic Coding Evals. Anthropic Engineering Blog, <https://www.anthropic.com/engineering/infrastructure-noise>. Accessed 14 September 2026.
- [44] SGLang contributors. 2026. [RFC] Agent-Aware KV Cache Phase 1 for Agentic Workloads (SGLang issue #24656). <https://github.com/sgl-project/sglang/issues/24656>. Accessed 14 September 2026.
- [45] SGLang contributors. 2026. [RFC] First-Class, Versioned KV Hint Envelope for SGLang (SGLang issue #36224). <https://github.com/sgl-project/sglang/issues/36224>. Accessed 14 September 2026.
- [46] Rana Shahout, Cong Liang, Shiji Xin, Qianru Lao, Yong Cui, Minlan Yu, and Michael Mitzenmacher. 2025. Fast Inference for Augmented Large Language Models. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc., 71562–71591. arXiv:2410.18248 doi:10.52202/085713-2404
- [47] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, USA, 965–988. <https://www.usenix.org/conference/osdi24/presentation/sheng>
- [48] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. 2025. The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems. *IEEE Transactions on Information Forensics and Security* 20 (2025), 11431–11446. doi:10.1109/TIFS.2025.3622954
- [49] SWE-agent contributors. 2025. mini-SWE-agent: The 100 Line AI Agent That Solves GitHub Issues. <https://github.com/SWE-agent/mini-swe-agent>. Accessed 14 September 2026.
- [50] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. 2026. CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents. arXiv:2606.16824 [cs.DC] doi:10.48550/arXiv.2606.16824
- [51] Dan Tsafir, Yoav Etsion, and Dror G. Feitelson. 2007. Backfilling Using System-Generated Predictions Rather than User Runtime Estimates. *IEEE Transactions on Parallel and Distributed Systems* 18, 6 (2007), 789–803. doi:10.1109/TPDS.2007.70606
- [52] vLLM project contributors. 2026. [RFC]: Context-Aware KV-Cache Retention API (Prioritized Evictions). vLLM issue #37003; implementation PR #38514, “[Feature] Context-Aware KV-Cache Retention API”. <https://github.com/vllm-project/vllm/issues/37003> Accessed 14 September 2026.
- [53] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR 2025)*. 65882–65919. https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html
- [54] Zhiyu Wang and Rajkumar Buyya. 2026. Preserving Admission Responsibility in Multi-Tenant Large Language Model Prefix Caches. arXiv:2608.01657 [cs.DC] doi:10.48550/arXiv.2608.01657
- [55] Alexander Wei and Fred Zhang. 2020. Optimal Robustness-Consistency Trade-offs for Learning-Augmented Online Algorithms. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 8042–8053. arXiv:2010.11443 [cs.LG] <https://proceedings.neurips.cc/paper/2020/hash/5bd844f11fa520d54fa5edec06ea2507->

Abstract.html

- [56] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. 2026. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. arXiv:2606.00866 [cs.OS] doi:10.48550/arXiv.2606.00866
- [57] Yechen Xu, Xinhao Kong, Tingjun Chen, and Danyang Zhuo. 2024. Conveyor: Efficient Tool-aware LLM Serving with Tool Partial Execution. arXiv:2406.00059 [cs.CL] doi:10.48550/arXiv.2406.00059
- [58] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601
- [59] Mingyan Yang, Guanjie Wang, Manqi Luo, Yifei Liu, Chen Chen, Han Zhao, Yu Feng, Quan Chen, and Minyi Guo. 2025. Justitia: Fair and Efficient Scheduling of Task-parallel LLM Agents with Selective Pampering. arXiv:2510.17015 [cs.LG] doi:10.48550/arXiv.2510.17015
- [60] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.). Curran Associates, Inc., 62557–62583. arXiv:2312.07104 [cs.AI] doi:10.52202/079017-2000
- [61] Kaiyu Zhou, Yongsen Zheng, Yicheng He, Meng Xue, Xueluan Gong, Yuji Wang, Xuanye Zhang, and Kwok-Yan Lam. 2026. Beyond Max Tokens: Stealthy Resource Amplification via Tool Calling Chains in LLM Agents. arXiv:2601.10955 [cs.CR] doi:10.48550/arXiv.2601.10955
- [62] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560 [cs.LG] <https://arxiv.org/abs/2606.30560>

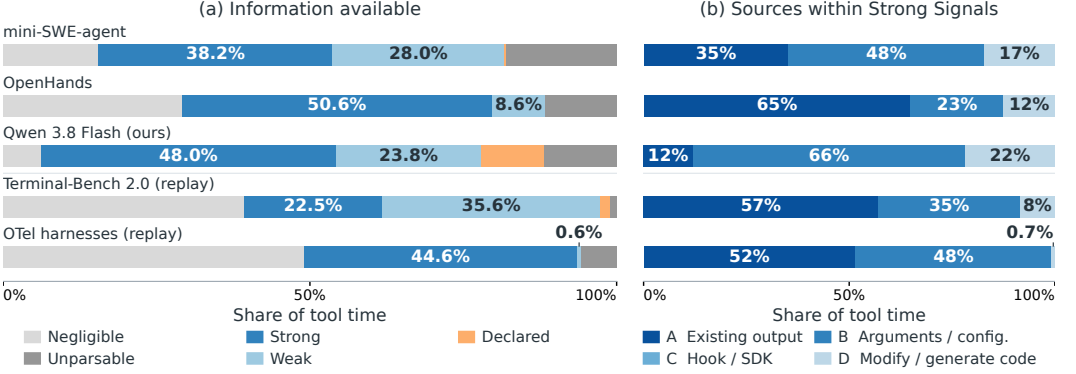


Fig. 11. The census by corpus. (a) The five kinds of Table 2 as shares of tool time. (b) Sources A–D within the strong kind, each bar normalized by its own strong time. The rows marked replay take their durations from a re-execution of the labelled calls in our sandbox environment.

A Census details

The census samples calls from four public corpora [17, 22, 30, 49, 53], stratified by source, scaffold, command class, and duration, and labels them with two frontier language models [38, 42]; a third model arbitrates their disagreements, and a human-labelled subset checks them.

Figure 11 repeats the census of Figure 4 for each corpus. The two SWE-bench corpora record their durations; Terminal-Bench [30] and the OpenTelemetry traces [17] do not, so their shares are of the time measured in a replay of the labelled calls. The corpora differ in what their tool time is made of. The mini-SWE-agent runs are the most mixed, with sizeable strong, weak and unparsable shares, and nearly half of their strong signal sits behind a switch (B), because the scaffold runs its calls silenced. OpenHands spends more of its time on negligible calls, views and edits, and most of its strong signal is printed by the tools as they are (A). Terminal-Bench holds the largest weak share of any corpus and the smallest strong share. The OpenTelemetry harnesses sit at the other end: half of their time is negligible and almost all of the rest is strong, split evenly between A and B. Source C is empty everywhere, and D, the agent’s own scripts, ranges from a fifth of the strong signal in our own runs to almost nothing in the OpenTelemetry traces.

B Harness details

Table 5 lists the instruments by layer of silence: what hides the signal, what the harness does about it, what the side channel receives, and what kind of signal results.

C Accuracy details

Error by load condition and history source. Figure 12 is the matrix behind the pooled numbers of Figure 7: the four load conditions of the running call against the four sources of history, with the progress stream in the last column. The progress stream is equally accurate on three of the four conditions. The fourth is the one where the load is removed mid-call: there the stream’s rate was learned under load, so at the midpoint it overestimates the remaining time badly until it has observed the new rate, and a windowed rate shortens that lag. Contention also pushes a set of calls that were short on the idle machine past the reload threshold, where no history exists.

Timely triggers by family. Figures 13 and 14 split the trigger curve of Figure 8 by family under the two recovery budgets. The instrumented scripts, whose loop iterations are uniform, trigger in time in half of the runs at either budget. Test suites, whose tests are uneven, trigger in time in a

What silences it	What the harness does	What the side channel gets	Verdict
<i>Layer 1: the infrastructure and the agent's habits</i>			
Bars turned off by environment variables	Re-enable the bars; strip them from the agent's stream	The bar's fraction and total	Weak for pip in a pipe
Trailing tail or head, > file	Copy the full stream to a side file before the filter	The full stream; not what a mid-pipe grep hides	Transport, not a signal
<i>Layer 2: tools that speak only when asked</i>			
setuptools build (build_ext)	Dry run for the unit count N ; verbose switch on, its lines hidden	k of N files; last-unit marker 2 s before the exit	Strong
Isolated build (pip install -e .)	Count object files as they appear; N from the extension list	k of N objects	Strong
pytest with -q	Parse the dot line as it grows; N from --collect-only; the flag kept	k of N tests; [100%] 0.2 s before the exit	Strong
Django's test runner	Parse the dot line; -v 2 on the side, folded back to dots	k tests; N only where Django prints it; end marker 2 s before the exit	Weak below Django 4.1, strong above
pip install from PyPI	--progress-bar raw; dry run for N and the byte total	Bytes over the total; install marker <1 s before the exit	Strong on a slow network, weak on an open one
make-style build	Verbose unit stream; make -n gives no N	Units without a total; last-unit marker	Weak
<i>Layer 3: the agent's own code</i>			
The agent's own script	A progress line added at generation time; original and instrumented runs compared	k of N loop iterations	Strong

Table 5. A sample of instruments, by layer of silence. What hides the signal, what the harness does about it, what the side channel receives, and what kind of signal results.

fifth to a third. Builds, whose compile steps are the most uneven, rarely trigger in time on a tight budget, but do so on most runs once more lead is allowed. The best baseline with its best history triggers in time on almost no run outside the scripts. Figure 15 relaxes the one-sided count to a two-sided one: a trigger counts when it lands within a tolerance of the ideal time, early or late. Within a tolerance equal to the recovery budget, the progress stream lands close to the ideal time on well over half of the runs with two seconds of recovery and on a third with half a second, where the baselines almost never do.

Trajectories after a load change. Figure 16 follows the online estimate across a load change on two real replays of the same editable install: a contending neighbour starts in one and stops in the other. The whole-run rate carries the old load for tens of seconds. A windowed rate recovers within about ten seconds, at the price of overshooting.

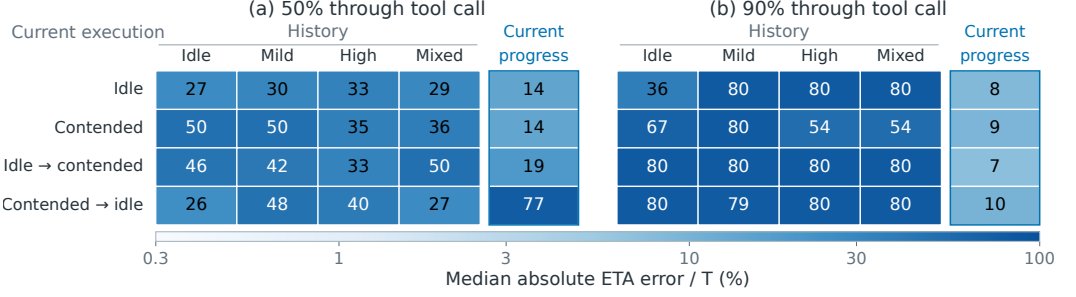


Fig. 12. Median absolute ETA error at 50 % and 90 % of the call, as a share of the call’s own duration, pooled over tests, builds and instrumented scripts. Rows are the load condition of the running call. Each history column shows the best method given the history source (idle, mild, high, mixed); the last column is the progress stream. Each call replayed under the four conditions.

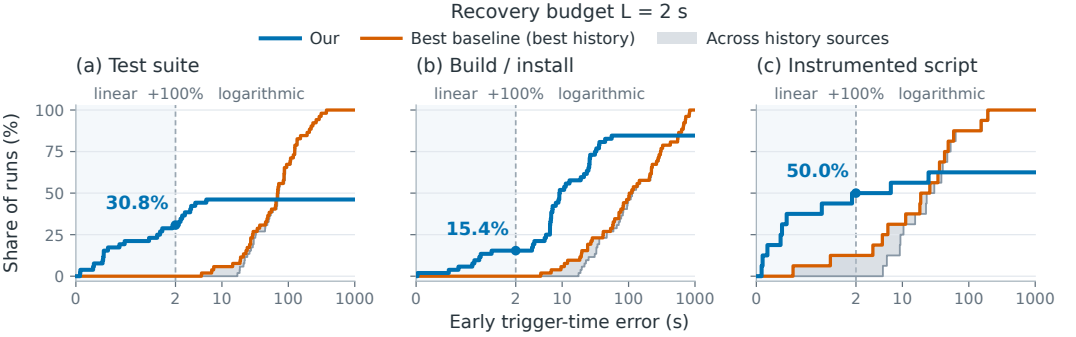


Fig. 13. Timely first triggers under a two-second recovery budget, by family: the share of runs whose first threshold crossing lands within the extra lead allowed, for the progress stream (blue) and the best baseline with its best history (orange); the grey band spans the four history sources. Late and never-triggered runs count in the denominator. At the marked budget, an extra lead equal to the recovery budget, the progress stream triggers in time in 31 % of test runs, 15 % of builds and 50 % of scripts; the best baseline in 0, 0 and 13 %.

D End-to-end details

The engine patch. The engine is vLLM [24] with prefix caching on; the host tier is its own KV offloading to DRAM. An unmodified engine gives an outside controller no handle on an idle context: keeping one meant sending it a one-token request, a recompute that competes with real turns, and letting one go was impossible. The patch adds one route that names a context by the id of an earlier response and offers three operations: *refresh* moves its resident blocks to the safe end of the eviction queue at no cost, *release* moves them to the front while keeping them addressable, and *query* reports how much of it is resident. The operations run between scheduler steps, skip blocks that a running request holds, and never raise into the serving loop. The scheduler, its eviction rule, and the model are untouched, and an engine that receives no hints behaves exactly as before. A hint costs one round trip of a few milliseconds, and the patch is a few hundred added lines with no existing function changed.

The controller. The controller runs beside the harness and reads each session’s progress stream. While the reported remaining time is under the keep threshold, it refreshes the session’s context; once the estimate has moved well past the threshold, it stops, and the context ages out under LRU

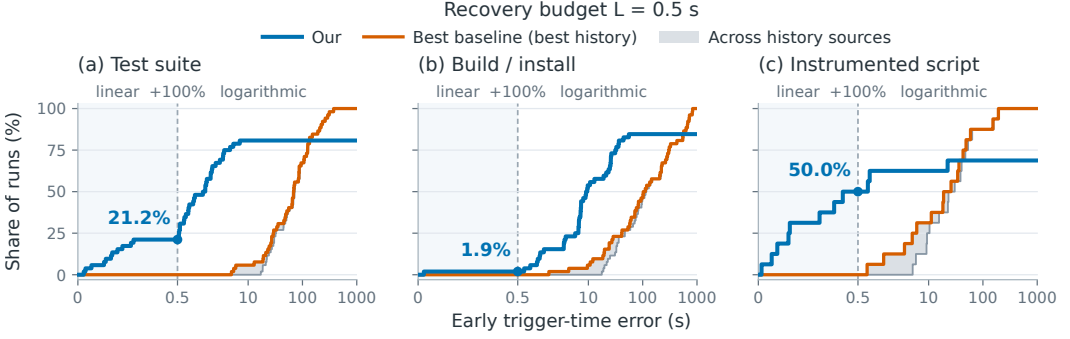


Fig. 14. The same breakdown under a half-second recovery budget. At the marked budget the progress stream triggers in time in 21 % of test runs, 2 % of builds and 50 % of scripts; no baseline triggers in time on any run.

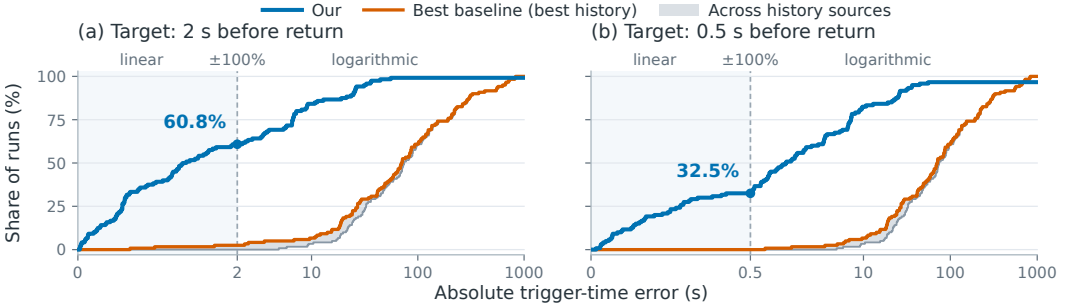


Fig. 15. Absolute first-trigger time error, all families pooled: the share of runs whose first trigger lands within a tolerance of the ideal trigger time, early or late, under a two-second (a) and a half-second (b) recovery budget. Never-triggered runs count in the denominator. At a tolerance equal to the recovery budget the progress stream is within it on 61 % and 33 % of runs; the best baseline with its best history on 3 % and 0 %.

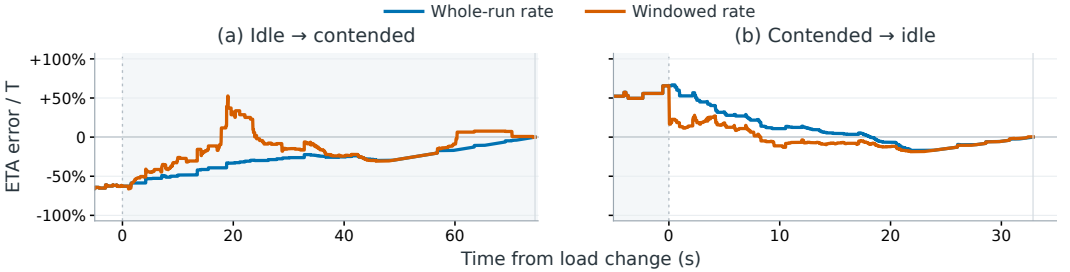


Fig. 16. Online ETA error across a load change on two real replays of the same editable install, read from verbose compile output with a known total: a contending neighbour starts (a) or stops (b) at time zero, and the shaded span is the contended interval. Both estimators are reconstructed from the events available at the time; a positive error overestimates the remaining time.

or moves to the host tier; if the estimate falls back, the refreshes resume. With a host tier, the controller also refreshes the context one lead time before the predicted return and reloads it if it has gone. The oracle runs the same controller with the true remaining time; LRU never hints.